



**Université Abdelmalek Essaâdi
Faculté des Sciences et Techniques de
Tanger**



Structures des données avancées et Théorie des graphes

Plan

› **Algorithmique**

- Algorithme et vocabulaire
- Structures algorithmiques

› **Structures des données**

- Structures des données simples
- Structures des données avancées
 - › Liste chaînées
 - › Piles et files
 - › Arbres et arbres binaires
 - › Graphes

› **Théorie des Graphes**

Algorithme et vocabulaire

Identification du Problème

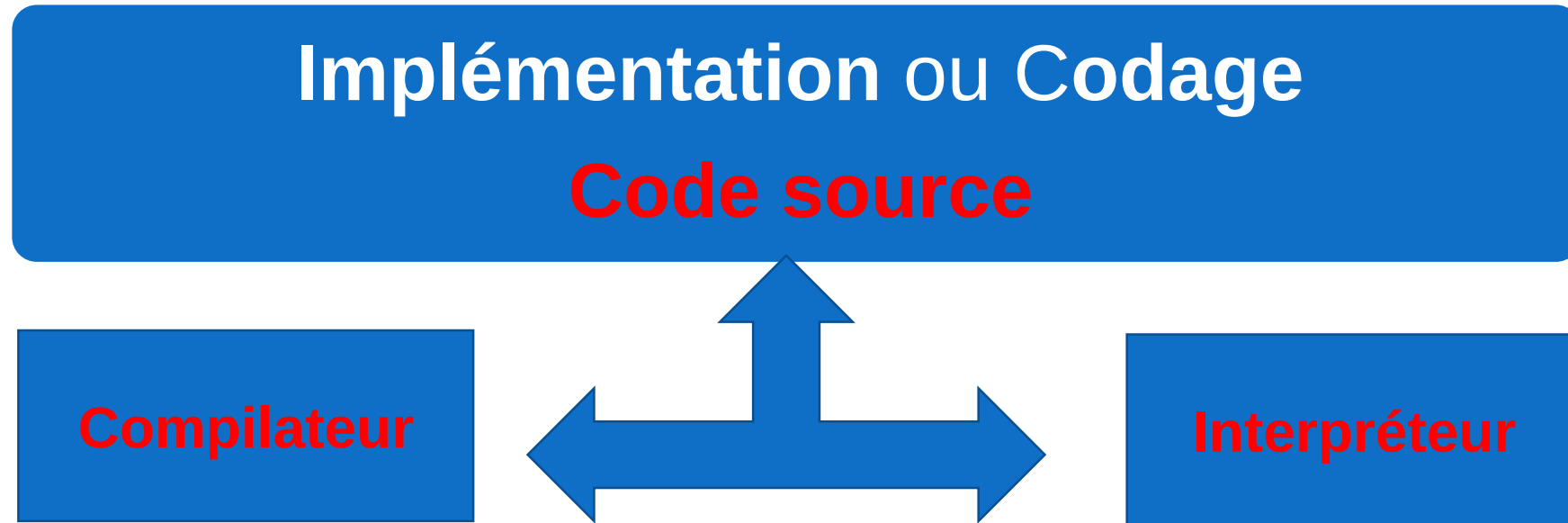
Algorithme

une succession d'opérations

Implémentation ou Codage

Code source

Algorithme et vocabulaire



- › Un compilateur traduit l'intégralité du code source d'un programme en un code machine ou en un code intermédiaire avant de l'exécuter
- › Un interpréteur lit et exécute le code source ligne par ligne ou instruction par instruction. Il ne génère pas de code machine indépendant.

Structures algorithmiques

- › les structures de données
 - Constantes
 - Variables
 - Tableaux
 - Structures simples
 - Structures récursives
(listes, Piles , Files , arbres , graphes)
- › Les structures de contrôle
 - Conditionnelles
 - de répétition

Les structures

Rappel

Les structures

- Les types composés
 - `typedef int entier ; /* on définit un nouveau type "entier" synonyme de "int" */`
 - `typedef int vecteur[3]; /* on définit un nouveau type "vecteur" synonyme de "tableau de 3 entiers" */`
- Exemple

```
include <stdio.h>
typedef int entier;
typedef float point[2];
void main()
{
    entier n = 6;
    point xy;
    xy[0] = 8.6;
    xy[1] = -9.45;
    etc ...
}
```

Les structures

- Les structures
 - Une structure est un ensemble de données de même type ou non.

1. Déclaration:

```
Struct nom_structure  
{  
  Type var1;  
  Type var2;  
};
```

```
typedef struct nom_structur  
{  
  type var1;  
  type var2;  
} nouveau_nom;
```

N.B. une structure peut contenir une autre structure

Les structures

› Exemple

```
struct point
{
    int x;
    int y;
    int z;
};
```

```
Typedef struct
{
    int x;
    int y;
    int z;
} point;
```

```
Typedef struct
{
    char nom[10];
    char prenom[10];
    int age;
    float note;
} fiche
```

Les structures

› Exercice d'application 1

- Ecrire un programme qui permet de définir une structure de données regroupant toutes les informations (nom, prenom, CIN, moyenne) pour plusieurs étudiants.
- ajouter une fonction qui permet de trouver l'étudiant qui a la plus faible moyenne.

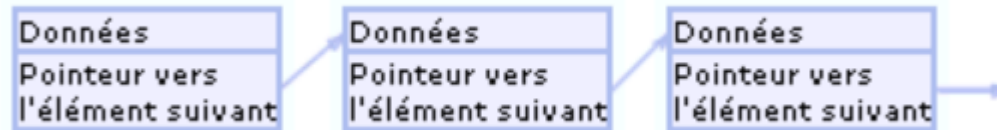
Listes chaînées

Rappel

Listes chaînées

› Définition

- › Une structure qui contient des données et un pointeur vers la structure suivante.
- › Une liste chaînée est un ensemble d'éléments (cellules) dont chaque cellule est composée des deux parties suivantes:
 - la valeur à stocker,
 - l'adresse de]



- › **N.B.** S'il n'y a plus d'élément suivant, alors l'adresse sera NULL, et désignera la fin de la chaîne.

Listes chaînées

› Syntaxe de Déclaration

– Exemple

```
Struct contact
{
    Char nom [20]
    Char tel[20]
    Struct contact *Csuivant
}
```

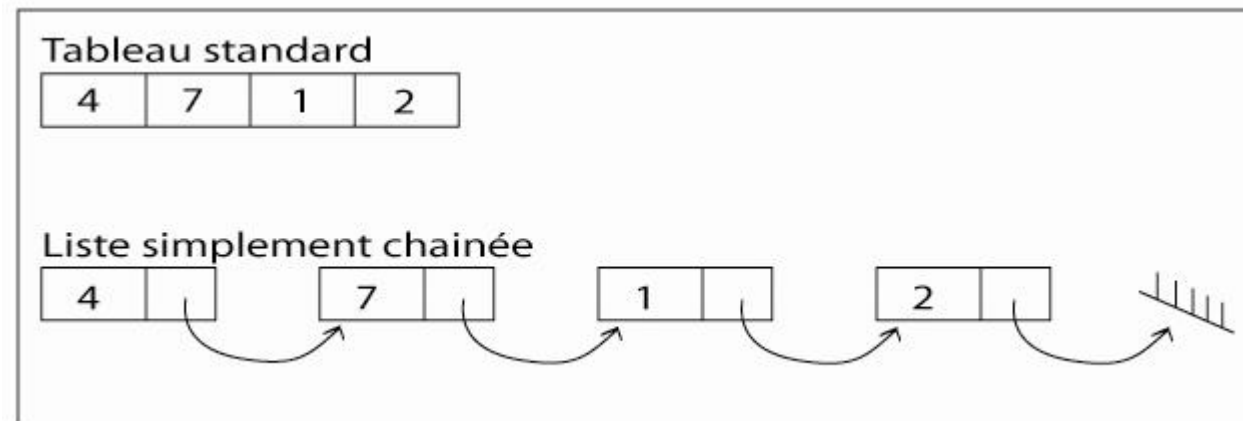
Listes chaînées

› Pour les listes chaînées

- Pour déclarer une liste chaînée, il suffit de créer le pointeur qui va pointer sur le premier élément de votre liste chaînée, **aucune taille n'est donc à spécifier**.
- La taille de la liste n'a pas de limites (ca dépend de la mémoire disponible)
- Pour accéder à l'élément i de la liste, il faut **traverser les $i-1$** éléments précédents.
- Il est possible d'ajouter, de supprimer et d'intervertir des éléments d'une liste chaînée en manipulant simplement leurs pointeurs.
- Lorsque la structure contient des données, un pointeur vers la structure suivante, et un pointeur vers la structure précédente on parle de **liste chaînée double**
- Lorsque la structure contient des données, un pointeur vers une première structure suivante, et un pointeur vers une seconde, on parle d'**arbre binaire**

Listes chaînées

› Différence entre tableau des structures et liste chaînées



Liste chaînée

› Fonctions autorisées

– Ajout

- › En tête
- › En fin
- › Après ou avant un élément

– Suppression

- › En tête
- › En fin
- › Un élément de la liste

– Affichage

– Recherche

›

Listes chaînées

1-Déclaration d'une liste chaînée

```
#include <stdlib.h>
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Listes chaînées

2-Initialisation d'une liste chaînée

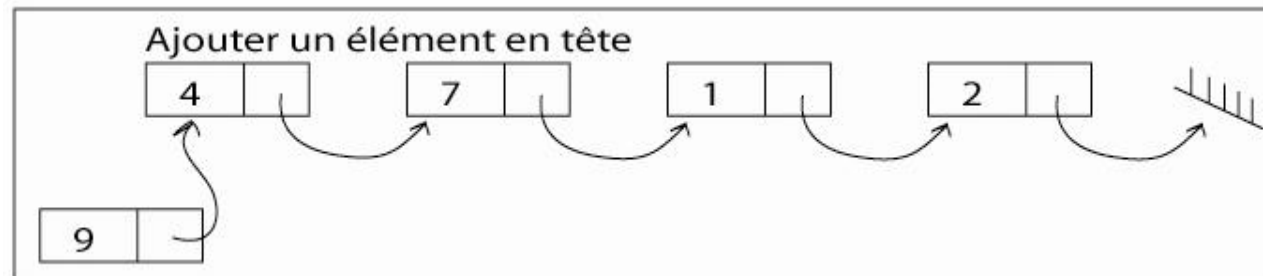
- › Pour déclarer une liste chaînée, il suffit d attribuer la valeur NULL au premier élément de la liste chaînée déclarée

```
int main()
{ /* Déclarons 3 listes chaînées de façons différentes mais équivalentes */
  list ma_liste1 = NULL;
  element *ma_liste2 = NULL;
  struct element *ma_liste3 = NULL;
  return 0;
}
```

Listes chaînées

3-Ajout d'un élément en tête d'une liste

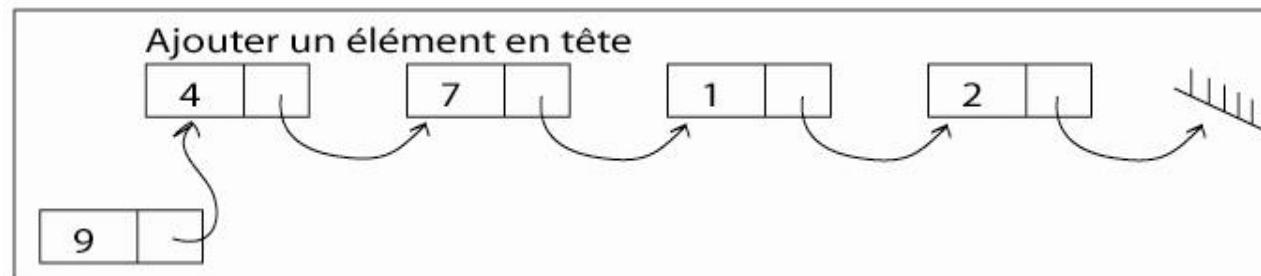
- › Ajouter en tête
- › Ajouter en fin
- › Ajouter avant ou après un élément



Listes chaînées

3-Ajout d'un élément en tête d'une liste

```
list ajouterEnTete(list liste, int valeur)
{
    /* On crée un nouvel élément */
    element* nouvelElement = (element*) malloc(sizeof(element));
    /* On assigne la valeur au nouvel élément */
    nouvelElement→val = valeur;
    /* On assigne l'adresse de l'élément suivant au nouvel élément */
    nouvelElement→nxt = liste;
    /* On retourne la nouvelle liste, i.e. le pointeur sur le premier élément */
    return nouvelElement;
}
```

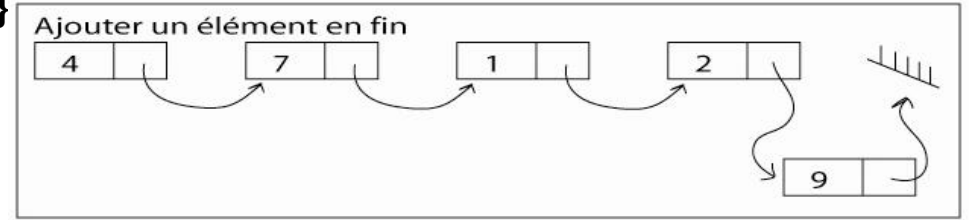


Listes chaînées

4-Ajout d'un élément en fin d'une liste

```
list ajouterEnFin(list liste, int valeur)
{
    /* On crée un nouvel élément */
    element* nouvelElement =
        (element*)malloc(sizeof(element));
    /* On assigne la valeur au nouvel élément */
    nouvelElement->val = valeur;
    /* On ajoute en fin, donc aucun élément ne va
    suivre */
    nouvelElement->nxt = NULL;
    if(liste == NULL)
    {
        /* Si la liste est vide il suffit de renvoyer l'élément
        créé */
        return nouvelElement;
    }
}
```

```
else
{ /* Sinon, on parcourt la liste à l'aide d'un pointeur
temporaire et on indique que le dernier élément de la
liste est relié au nouvel élément */
    element *temp=liste;
    while(temp->nxt != NULL)
    {
        temp = temp->nxt;
    }
    temp->nxt = nouvelElement;
    return liste;
}
```



Listes chaînées

► Exercice d'application 2

- Ecrire la fonction permettant d'afficher les éléments d'une liste

► Solution

```
void afficherListe (list liste)
{
    element *tmp = liste;
    /* Tant que l'on n'est pas au bout de la liste */
    while(tmp != NULL)
    {
        /* On affiche */
        printf("%d ", tmp->val);
        /* On avance d'une case */
        tmp = tmp->nxt;
    }
}
```

Listes chaînées

5-Suppression d'un élément en tête d'une liste

- › Supprimer en tête
- › Supprimer en fin
- › Supprimer un élément

Listes chaînées

5-Suppression d'un élément en tête d'une liste

```
list supprimerElementEnTete(list liste)
{
    if(liste != NULL)
    {
        /* Si la liste est non vide, on se prépare à renvoyer l'adresse de l'élément en 2ème position*/
        element* aRenvoyer = liste->nxt;
        /* On libère le premier élément */
        free(liste);
        /* On retourne le nouveau début de la liste */
        return aRenvoyer;
    }
    else
    {
        return NULL;
    }
}
```

Listes chaînées

6-Suppression d'un élément en fin d'une liste

```
list supprimerElementEnFin(list liste)
{
    /* Si la liste est vide, on retourne NULL */
    if(liste == NULL)
        return NULL;
    /* Si la liste contient un seul élément */
    if(liste->nxt == NULL)
    {
        /* On le libère et on retourne NULL (la liste est
        maintenant vide) */
        free(liste);
        return NULL;
    }
    /* Si la liste contient au moins deux éléments */
    element* tmp = liste;
    element* ptmp = liste;
```

```
    /* Tant qu'on n'est pas au dernier élément */
    while(tmp->nxt != NULL)
    {
        /* ptmp stock l'adresse de tmp */
        ptmp = tmp;
        /* On déplace tmp (mais ptmp garde l'ancienne valeur de
        tmp */
        tmp = tmp->nxt;
    }
    /* A la sortie de la boucle, tmp pointe sur le dernier élément,
    et ptmp sur l'avant-dernier. On indique que l'avant-dernier
    devient la fin de la liste et on supprime le dernier élément */
    Ptmp->nxt = NULL;
    free(tmp);
    return liste;
}
```

Listes chaînées

7-Recherche d'un élément dans une liste

```
list rechercherElement(list liste, int valeur)
{
    element *tmp=liste;
    /* Tant que l'on n'est pas au bout de la liste */
    while(tmp != NULL)
    {
        if(tmp->val == valeur)
        {
            /* Si l'élément a la valeur recherchée, on renvoie son
            adresse */
            return tmp;
        }
        tmp = tmp->nxt;
    }
    return NULL;
}
```

Listes chaînées

► Exercice d'application

- Ecrire une fonction permettant de compter l'occurrence d'une valeur dans une liste

► Solution

```
int nombreOccurences(lliste liste, int valeur)
{
    int i = 0;
    /* Si la liste est vide, on renvoie 0 */
    if(liste == NULL)
        return 0;
    /* Sinon, tant qu'il y a encore un élément ayant la val = valeur */
    while((liste = rechercherElement(liste, valeur)) != NULL)
    {
        /* On incrémente */
        liste = liste->nxt;
        i++;
    }
    /* Et on retourne le nombre d'occurrences */
    return i;
}
```

Piles et Files

- › Les piles et files sont des structures des données particulières.
- › Ces deux variantes peuvent être représentées par des **structures statiques** ou **dynamiques**.
- › Elles permettent de contrôler la manière dont les nouveaux éléments sont ajoutés.

Piles et Files

› Définition

- Une pile est une structure de données Simple .
- Le principe des piles en programmation est de stocker des données au fur et à mesure les unes au-dessus des autres pour pouvoir les récupérer.
- On peut implémenter une pile dans un tableau (**pile statique**) ou dans une liste chaînée (**pile dynamique**).
- Il s'agit d'une structure de type **LIFO** (Last In First Out).

Le principe de fonctionnement d'une Pile : le dernier inséré est le premier retiré alors :

- **On ajoute et on retire d'en fin**
- **On ajoute et retire d'en tête**

Piles et Files

› Syntaxe de déclaration

```
typedef struct entiere  
{  
    Int Valeur;  
    struct entiere *suivant;  
  
};  
typedef entiere * Pile
```

- Nom des éléments de la pile : entiere
- Nom du pointeur Sommet (Tête de la pile) : Pile

› Initialisation

```
Pile *p=NULL ;
```

Piles et Files

› Fonctions autorisées

- **Empiler (ajouter en tête)**, toujours au sommet, et jusqu'à la limite de la mémoire,
- **Dépiler (retirer ou supprimer en tête)**, toujours au sommet, si la pile n'est pas vide.

Piles et Files

› Empiler : Ajouter un élément

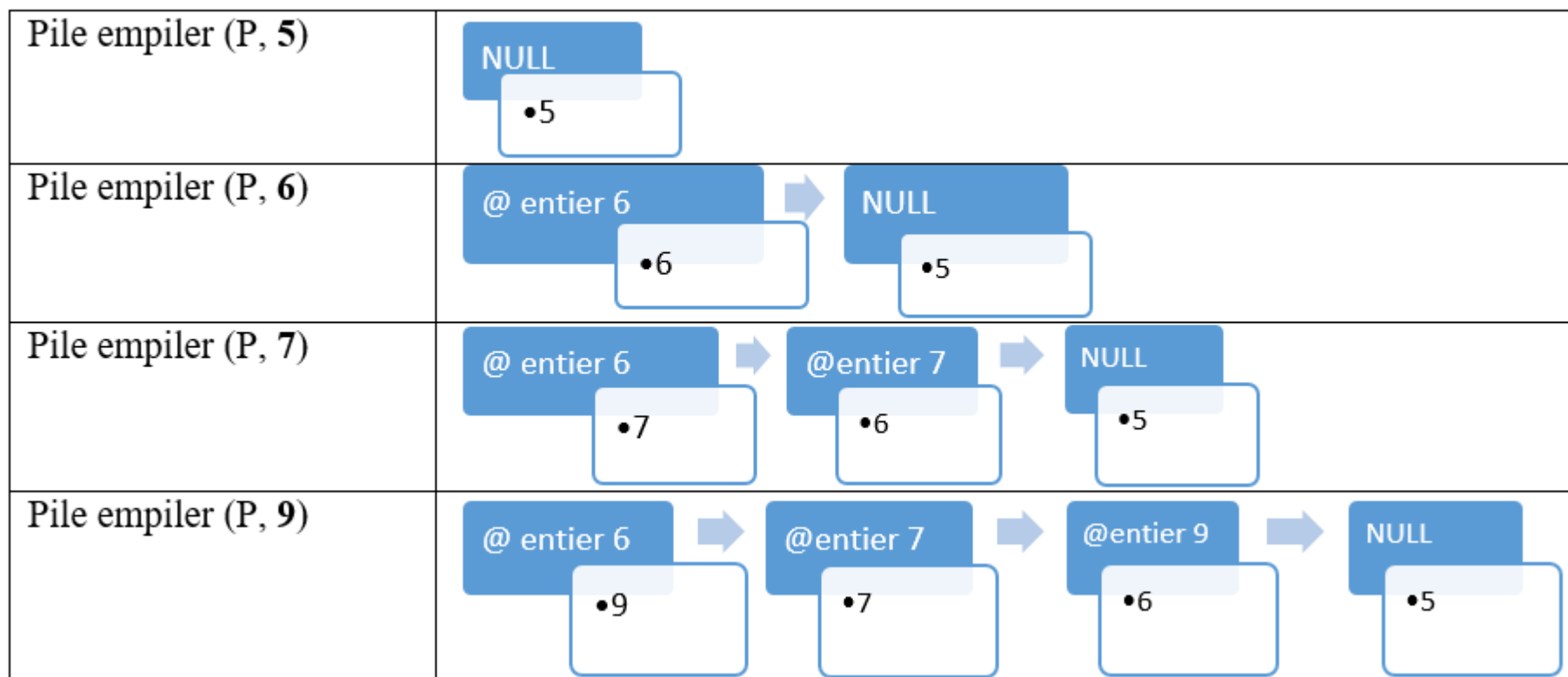
- L'ajout d'un élément dans la pile se fait en cinq étapes :
 1. On crée un nouvel élément de type **Pile**
 2. On assigne à la donnée de cet élément, la donnée que l'on veut ajouter.
 3. On fait pointer cet élément sur le sommet de la pile.
 4. On fait pointer le sommet de la pile sur cet élément.
 5. On retourne le nouvel sommet.

```
Pile empiler (Pile P, int valeur)
{
    Pile nouveau=(Pile)malloc(sizeof(entier));
    nouveau→val=valeur;
    nouveau→suivant=P;
    return nouveau;
}
```

Piles et Files

› Exemple

- **Empiler : Insérer des éléments dans la pile P**



Piles et Files

› Dépiler : Retirer un élément de la pile

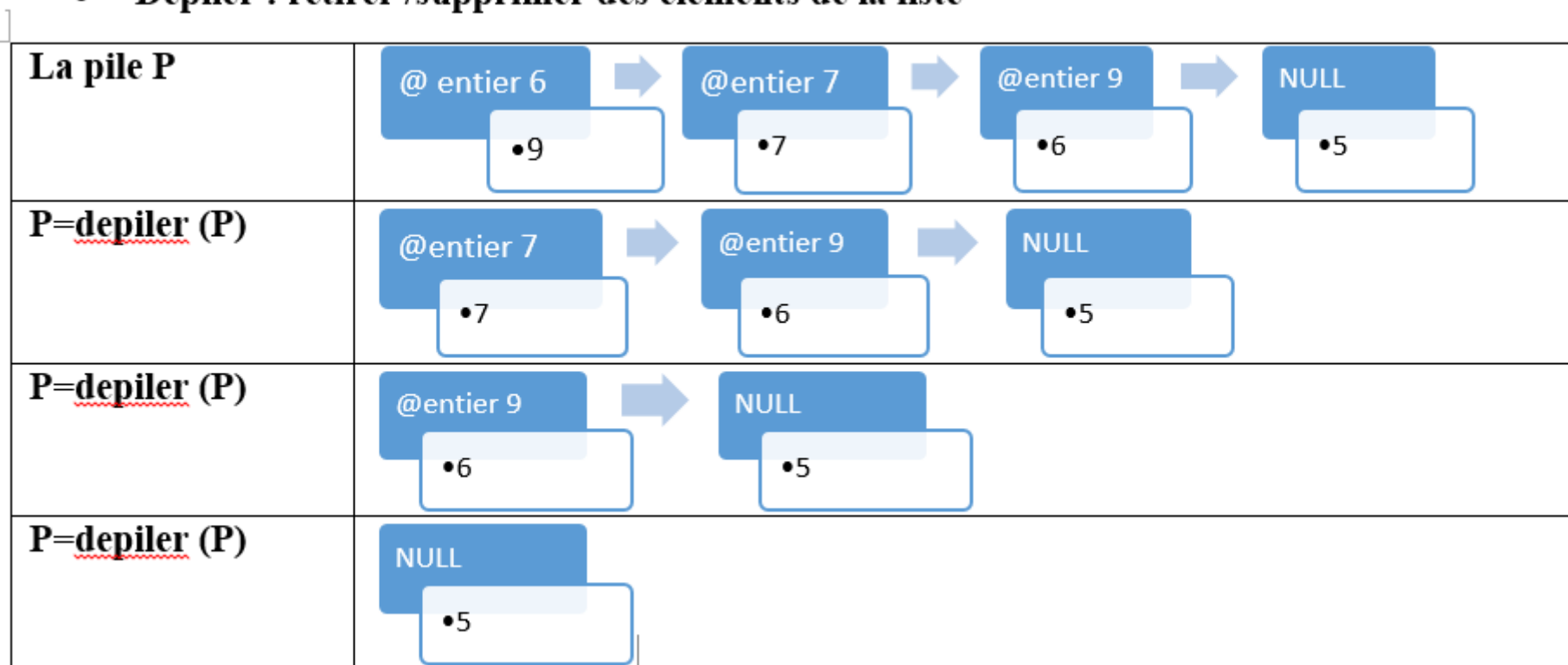
- Pour retirer un élément de la pile il faut tout d'abord :
 1. Vérifier si la pile n'est pas vide
 2. Si elle ne l'est pas, Stocker dans un élément temporaire l'adresse du deuxième élément de la pile
 3. Supprimer le premier élément
 4. Faire pointer la pile vers notre élément temporaire
 5. Retourner la nouvelle pile
 6. Sinon retourner la pile vide.

```
Pile depiler(Pile P)
{
    Pile NewSommet;
    if(P!=NULL)
    {
        NewSommet=P→suivant;
        free(P);
        return(NewSommet);
    }
    else
        return NULL;
}
```

Piles et Files

› Exemple

- **Dépiler : retirer /supprimer des éléments de la liste**



Piles et Files

› Afficher une pile

```
void afficherPile(Pile *pile)
{
    if (pile != NULL)
    {

        entier *tmp = pile;
        while (tmp != NULL)
        {
            printf("%d\n", tmp->valeur);
            tmp = tmp->suivant;
        }

        printf("\n");
    }
}
```

Piles et Files

› Définition

- Une file est une liste chaînée d'informations dans laquelle la récupération des données sera faite dans l'ordre d'insertion
- La file est une structure de données, qui permet de stocker les données dans l'ordre FIFO (First In First Out)

Le principe de fonctionnement d'une file : le premier inséré est le premier retiré alors :

- On ajoute en tête et on retire en fin
- On ajoute en fin et retire d'en tête

Piles et Files

› Syntaxe de déclarations

```
struct entier  
{  
    int valeur;  
    struct entier *suivant;  
};  
typedef entier * File;
```

› Initialisation

```
File *f=NULL ;
```

Piles et Files

› Fonctions autorisées

- **Enfiler (ajouter en fin)**, toujours en fin , et jusqu'à la limite de la mémoire,
- **Défiler (retirer ou supprimer en tête)**, toujours au sommet, si la pile n'est pas vide

Piles et Files

– Empiler : Ajouter un élément

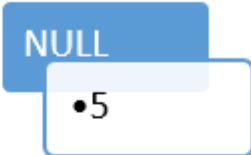
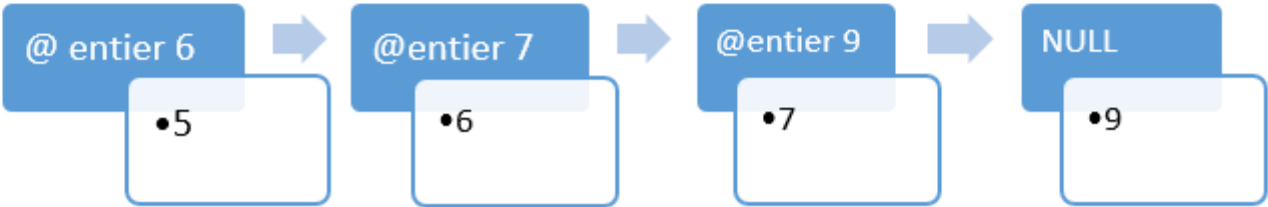
- › Enfiler c'est la fonction d'Insertion d'un élément en fin de la liste.
- › Si la liste est vide la nouvelle tête sera cet élément.
- › Si la file n'est pas vide,
 - il faut parcourir toute la file en partant du premier élément jusqu'à arriver au dernier.
 - On rajoutera notre nouvel élément après le dernier.

```
void enfiler(File *file, int nvNombre)
{
    entier *nouveau = malloc(sizeof(*entier));
    nouveau->valeur = nvNombre;
    nouveau->suivant = NULL;
    if (tmp != NULL) /* La file n'est pas vide */
    {
        /* On se positionne à la fin de la file */
        Entier *tmp = file;
        while (tmp->suivant != NULL)
        {
            tmp = tmp->suivant;
        }
        tmp->suivant = nouveau;
    }
}
```

Piles et Files

› Exemple

- **Efiler** : Insérer des éléments dans la pile P

F=enfiler (F, 5)	
F=enfiler (F, 6)	
F=enfiler (F, 7)	
F=enfiler (F, 9)	

Piles et Files

› Défiler : Retirer un élément (supprimer de la tête)

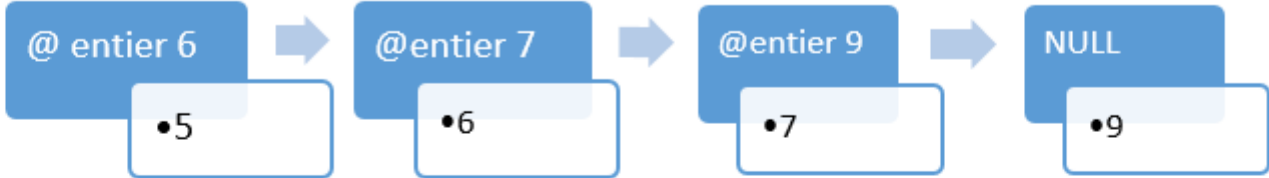
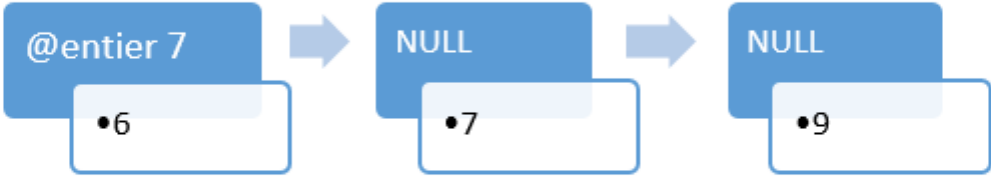
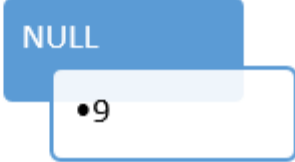
- Si la liste est vide, on retourne la valeur Null
- Sinon on Supprime la tête de la liste et on renvoie sa valeur.
- La nouvelle tête de la liste sera l'élément qui suit l'ancienne tête.

```
int defiler (file F)
{
    if(F==NULL)
        return (NULL);
    else
    {
        file *NewF;
        NewF=F;
        F=F→suivant;
        V= NewF→valeur;
        free (NewF);
    }
    return(v);
}
```

Piles et Files

› Exemple

- Défiler : retirer /supprimer des éléments de la liste

La file F	
<u>defiler</u> (F)	
<u>defiler</u> (F)	
<u>defiler</u> (F)	

Piles et Files

› Exercice d'application

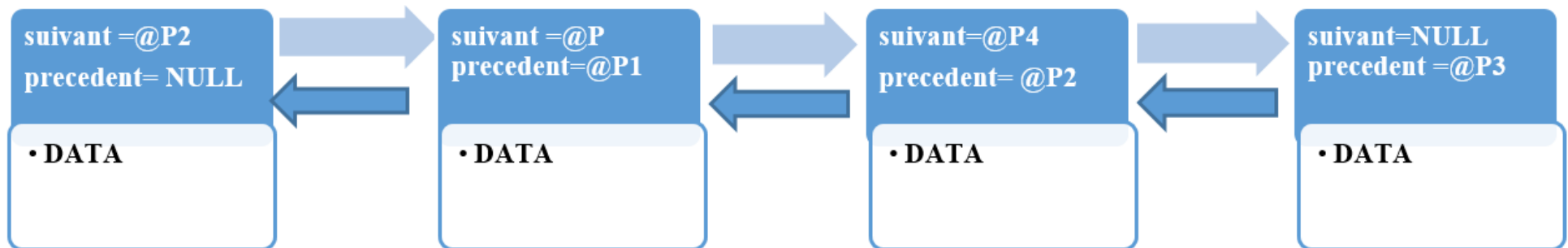
- Un palindrome est une chaîne de caractères qui se lit de la même manière de gauche à droite et de droite à gauche.
- Ecrire un programme en C qui détermine si une chaîne de caractères est un palindrome.

Liste doublement chaînée

Liste doublement chaînée

› Définition

- Une liste **doublement chaînée** est une structure de données contenant :
 - › Des données.
 - › Un pointeur vers l'élément suivant (NULL si l'élément suivant n'existe pas)
 - › Un pointeur vers l'élément précédent (NULL si l'élément précédent n'existe pas)



Liste doublement chaînée

› Syntaxe de déclaration

```
// Structure d'un nœud de la liste doublement chaînée  
typedef struct Node {  
    char data;  
    struct Node *prev;  
    struct Node *next;  
} Node;
```

```
// Structure pour stocker le début et la fin de la liste  
typedef struct List {  
    Node *head;  
    Node *tail;  
} List;
```

Liste doublement chaînée

› Initialisation

```
// Fonction pour initialiser une liste doublement chaînée vide  
List initializeList() {  
    List list;  
    list.head = NULL;  
    list.tail = NULL;  
    return list;  
}
```

Liste doublement chaînée

› Fonctions autorisées

– Ajout

- › En tête
- › En fin
- › Après ou avant un élément

– Suppression

- › En tête
- › En fin
- › Un élément de la liste

– Affichage

– Recherche

›

Liste doublement chaînée

› Insertion au début de la liste

```
void prepend(List *list, char data) {  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    if (newNode == NULL) {  
        perror("Échec de l'allocation de mémoire pour le nœud");  
        exit(EXIT_FAILURE);  
    }  
    newNode->data = data;  
    newNode->prev = NULL;  
    newNode->next = list->head;  
  
    if (list->head == NULL) {  
        list->tail = newNode;  
    } else {  
        list->head->prev = newNode;  
    }  
  
    list->head = newNode;  
}
```

Liste doublement chaînée

› Insertion en fin de la liste

```
void append(List *list, char data) {  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    if (newNode == NULL) {  
        perror("Échec de l'allocation de mémoire pour le nœud");  
        exit(EXIT_FAILURE);  
    }  
    newNode->data = data;  
    newNode->prev = list->tail;  
    newNode->next = NULL;  
  
    if (list->tail == NULL) {  
        list->head = newNode;  
    } else {  
        list->tail->next = newNode;  
    }  
  
    list->tail = newNode;  
}
```

Liste doublement chaînée

› Insérer avant un élément

```
void insertBefore(List *list, char dataToInsertBefore,
char data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL)
    {
        perror("Échec de l'allocation de mémoire pour
le nœud");
        exit(EXIT_FAILURE);
    }
    newNode->data = data;

    Node *current = list->head;
    while (current != NULL) {
        if (current->data == dataToInsertBefore)
```

```
{
    newNode->prev = current->prev;
    newNode->next = current;
    current->prev = newNode;

    if (newNode->prev != NULL) {
        newNode->prev->next = newNode;
    }

    if (current == list->head) {
        list->head = newNode;
    }

    return;
}
current = current->next;
}
}
```

Liste doublement chaînée

› Insérer après un élément

```
void insertAfter(List *list, char dataToInsertAfter,
char data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL) {
        perror("Échec de l'allocation de mémoire pour le
nœud");
        exit(EXIT_FAILURE); }
    newNode->data = data;
    Node *current = list->head;
    while (current != NULL)
    {
        if (current->data == dataToInsertAfter)
        {
            newNode->next = current->next;
            newNode->prev = current;
```

```
current->next = newNode;
    if (newNode->next != NULL)newNode->next->prev =
newNode;
    }
    if (current == list->tail)
    {
        list->tail = newNode;
    }

    return;
    }
    current = current->next;
}
}
```

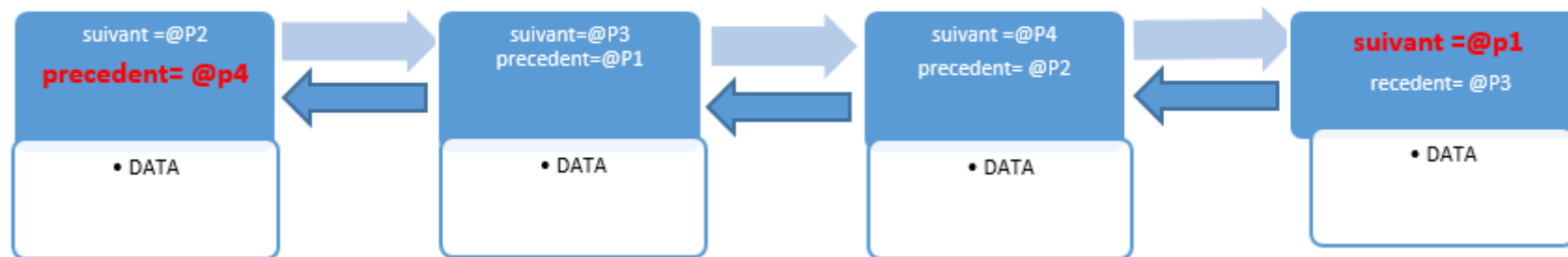
› Exercice d'application

- Développer un code en C pour
 1. Supprimer un élément au début
 2. Supprimer un élément en fin
 3. Rechercher un élément
 4. Afficher la liste

Liste circulaire

Liste circulaire

- › Une liste est dite circulaire si le dernier élément de cette liste remet au premier élément de cette liste ; c.-à-d. le pointeur suivant du dernier élément pointera sur le premier élément de la liste au lieu de la valeur NULL.
- › L'absence du NULL dans ce type de liste implique des conditions d'arrêt en rapport avec la **tête de liste ou racine de liste**.



Liste circulaire

- › La liste circulaire est une sorte de liste simplement ou doublement chaînée, qui comporte une caractéristique supplémentaire pour le déplacement dans la liste.
- › Dans une liste circulaire tous les éléments sont accessibles à partir de n'importe quel autre nœud.
- › Une liste circulaire n'a pas de premier et de dernier nœud.
- › Par ailleurs, un pointeur NULL signifie une liste vide.
- › Le parcours se fait à partir de n'importe quelle adresse contenue dans le pointeur courant jusqu'à retrouver cette même adresse.
- › La concaténation de deux listes circulaires peut se faire sans avoir à parcourir les deux listes

Liste circulaire

- › Les différences avec une liste simple (ou double) non circulaire sont :
 - L'ajout se fait après ou avant l'adresse contenue dans le pointeur courant et non au début ou à la fin.
 - La suppression également.
 - Le parcours se fait à partir de n'importe quelle adresse contenue dans le pointeur courant jusqu'à retrouver cette même adresse.

Liste circulaire

› Syntaxe de déclaration

// liste circulaire simplement chaînée

```
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

// liste circulaire doublement chaînée

```
typedef struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
} Node;
```

Liste circulaire

› Initialisation

```
// liste circulaire simplement chaînée
Node *initializeCircularList()
{
    return NULL;
}

// liste circulaire doublement chaînée

Node *initializeCircularDoubleList() {
    return NULL;
}
```

Liste circulaire

- › Fonctions autorisées
 - Ajout
 - › Apres ou avant un élément
 - Suppression
 - Affichage
 - Recherche

›

Liste circulaire

› Ajout

// liste circulaire simplement chaînée

```
Node *addToCircularList(Node *head, int data) {  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (head == NULL) {  
        newNode->next = newNode; // La liste est vide, le  
nœud pointe vers lui-même  
        return newNode;  
    }  
  
    Node *current = head;  
    while (current->next != head) {  
        current = current->next;  
    }  
  
    current->next = newNode;  
    newNode->next = head; // Le dernier nœud pointe  
vers la tête pour former une boucle  
    return head;  
}
```

// liste circulaire doublement chaînée

```
Node *addToCircularDoubleList(Node *head, int data)  
{  
    Node *newNode = (Node *)malloc(sizeof(Node));  
    newNode->data = data;  
  
    if (head == NULL) {  
        newNode->prev = newNode;  
        newNode->next = newNode;  
        return newNode;  
    }  
  
    Node *tail = head->prev;  
    newNode->prev = tail;  
    newNode->next = head;  
    head->prev = newNode;  
    tail->next = newNode;  
  
    return head;  
}
```